이 파트에서는 **인공지능(AI)**의 기본 개념과 역사를 학습하고, 실습을 위한 **Python 개발환경**을 구축합니다.

이후 파트에서 본격적인 딥러닝과 생성형 AI를 다루기 전에, 머신러닝의 세 가지 핵심 학습 방법인 **지도학습**, **비지도학습**, **강화학습**의 원리와 차이점을 이해합니다.

학습 목표

- Python 개발환경(가상환경, VS Code)을 설치하고 설정할 수 있습니다.
- 인공지능의 정의, 분류, 발전 역사를 설명할 수 있습니다.
- 지도학습의 분류/회귀 문제를 이해하고 scikit-learn으로 모델을 만들 수 있습니다.
- 비지도학습(클러스터링, 차원축소)과 강화학습의 개념을 설명할 수 있습니다.

이 파트의 구성

챕터	제목	핵심 내용
Chapter 1	개발환경 구축과 인공지능 소개	Python, VS Code, 가상환경 설정 + AI 개념과 역사
Chapter 2	지도학습 (Supervised Learning)	분류, 회귀, 모델 평가, 과적합
Chapter 3	비지도학습과 강화학습	클러스터링, 차원축소, Q-Learning

이 챕터에서는 본격적인 AI 개발에 앞서 필요한 환경을 설정하고, 인공지능이 무엇인지 개념적으로 이해하는 시간을 갖습니다. 처음 접하는 분들도 단계별 안내를 따라가면 누구나 개발 환경을 완성할 수 있습니다.

학습 목표

- Python을 설치하고 버전을 관리하는 방법을 익힙니다.
- VS Code를 설치하고 개발에 필요한 기본 설정을 완료합니다.
- 가상환경을 생성하고 필수 라이브러리를 설치합니다.
- 인공지능의 정의, 분류, 역사를 이해합니다.
- 첫 번째 Python 프로그램을 직접 실행해봅니다.

1.1 이 책의 전체 로드맵과 학습 목표

이 책은 AI를 처음 접하는 분들이 기초 개념부터 실전 활용까지 단계적으로 성장할 수 있도록 설계되었습니다. 무작정 코드를 따라치는 것이 아니라 "왜 이렇게 하는가"를 이해하면서 학습하는 것이 목표입니다.

전체 본문은 9개 Part, 26개 Chapter로 구성되어 있으며, 흐름상 크게 세 단계로 나눌 수 있습니다.

- **기초 (Part 1~2 / Chapter 1~6)**: 개발환경, Python·데이터 기초, 머신러닝 알고리즘, 딥러닝 기초(CNN/RNN)
- **생성형 AI (Part 3~5 / Chapter 7~15)**: Transformer·LLM, 프롬프트 엔지니어링, OpenAI API 활용
- **실전 응용 (Part 6~9 / Chapter 16~26)**: 외부 서비스 연동, RAG, 웹 애플리케이션, 오픈소스·로컬 LLM, 종합 프로젝트

> 📌 ***학습 팁**: 본문을 따라가면서 반드시 직접 코드를 실행해보세요. 눈으로 읽는 것과 손으로 타이핑하는 것은 기억에 남는 양이 전혀 다릅니다.*

각 챕터는 이론 설명 → 코드 예시 → 실습 순서로 구성되어 있으며, 챕터 끝에는 핵심 내용을 정리한 요약이 제공됩니다.

1.2 Python 설치 및 버전 관리

Python(파이썬)은 현재 AI/머신러닝 분야에서 가장 널리 사용되는 프로그래밍 언어입니다. 문법이 간결하고 풍부한 라이브러리 생태계 덕분에 초보자도 빠르게 생산적인 코드를 작성할 수 있습니다.

Python 버전 선택

AI/데이터 분야에서는 **Python 3.10 이상** 버전을 권장합니다. Python 2.x는 2020년에 공식 지원이 종료되었으므로 반드시 Python 3.x를 설치해야 합니다.

> 💡 ***버전 권장**: 2026년 현재 NumPy 2.x · PyTorch 2.5+ · TensorFlow 2.18+ 등 주요 라이브러리가 Python 3.12까지 안정 지원합니다. 본문 실습은 **Python 3.11 또는 3.12** 환경에서 검증되었으며, 어느 쪽을 사용해도 무방합니다. Python 3.13은 일부 ML 라이브러리(특히 GPU 가속 빌드)가 늦게 지원되는 경우가 있어, 처음이라면 3.11/3.12를 추천합니다.*

Windows에서 Python 설치

1. https://www.python.org/downloads/ 에 접속합니다.
2. "Download Python 3.11.x" 버튼을 클릭하여 설치 파일을 내려받습니다.
3. 설치 파일을 실행하고, **반드시** 하단의 "Add Python to PATH" 체크박스를 선택합니다.
4. "Install Now"를 클릭하여 설치를 완료합니다.

설치가 완료되면 터미널(명령 프롬프트 또는 PowerShell)을 열고 아래 명령어로 설치를 확인합니다.

```
python --version
```

다음과 같이 버전 정보가 출력되면 정상적으로 설치된 것입니다.

```
Python 3.11.8
```

macOS에서 Python 설치

macOS에서는 **Homebrew**를 이용하는 방법이 가장 편리합니다. 터미널을 열고 아래 명령어를 순서대로 실행합니다.

```
# Homebrew 설치 (이미 설치된 경우 생략)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Python 3.11 설치
brew install python@3.11
```

pyenv로 버전 관리하기

프로젝트마다 다른 Python 버전이 필요한 경우 **pyenv(파이엔브)**를 사용하면 여러 버전을 손쉽게 관리할 수 있습니다.

```
# pyenv 설치 (macOS/Linux)
curl https://pyenv.run | bash

# 특정 버전 설치
pyenv install 3.11.8

# 전역 기본 버전 설정
pyenv global 3.11.8

# 현재 디렉토리에서만 특정 버전 사용
pyenv local 3.10.14
```

> 💡 ***참고**: Windows에서는 `pyenv-win` 을 사용할 수 있습니다. https://github.com/pyenv-win/pyenv-win 에서 설치 방법을 확인하세요.*

1.3 VS Code 설치 및 기본 설정

VS Code(Visual Studio Code)는 Microsoft에서 개발한 무료 코드 편집기로, Python 개발에 가장 널리 사용됩니다. 가볍고 빠르면서도 다양한 확장 기능을 통해 강력한 개발 환경을 구성할 수 있습니다.

VS Code 설치

1. https://code.visualstudio.com/ 에 접속합니다.
2. 운영체제에 맞는 설치 파일을 내려받아 실행합니다.
3. 설치 옵션에서 "PATH에 추가"와 "코드로 열기 추가" 항목을 선택합니다.

필수 확장 프로그램 설치

VS Code를 실행한 후 왼쪽 사이드바의 확장(Extensions) 아이콘을 클릭하고 아래 확장 프로그램을 검색하여 설치합니다.

- **Python** (Microsoft 제공): Python 언어 지원, 자동완성, 디버깅 기능
- **Pylance**: 타입 검사 및 고급 자동완성

- **Jupyter**: Jupyter Notebook 지원
- **indent-rainbow**: 들여쓰기를 색상으로 구분하여 가독성 향상
- **Korean Language Pack for VS Code**: 한국어 인터페이스 지원 (선택 사항)

Python 인터프리터 설정

VS Code에서 사용할 Python 인터프리터를 지정해야 합니다. 아래 단계를 따릅니다.

1. `Ctrl+Shift+P` (macOS: `Cmd+Shift+P`)를 눌러 명령 팔레트를 엽니다.
2. `Python: Select Interpreter`를 검색하여 선택합니다.
3. 목록에서 설치한 Python 버전을 선택합니다.

> 📌 ***확인 방법**: VS Code 하단 상태 표시줄(Status Bar)에 선택된 Python 버전이 표시됩니다. 예: `Python 3.11.8 64-bit`*

기본 설정 (settings.json)

`Ctrl+Shift+P` → `Preferences: Open User Settings (JSON)`을 열고 아래 설정을 추가하면 Python 개발에 최적화된 환경이 만들어집니다.

```
{
    "editor.fontSize": 14,
    "editor.tabSize": 4,
    "editor.insertSpaces": true,
    "editor.formatOnSave": true,
    "python.languageServer": "Pylance",
    "python.analysis.typeCheckingMode": "basic",
    "files.autoSave": "afterDelay",
    "files.autoSaveDelay": 1000
}
```

1.4 가상환경(venv) 생성과 관리

가상환경(Virtual Environment)은 프로젝트별로 독립된 Python 실행 환경을 만드는 기술입니다. 프로젝트마다 필요한 라이브러리의 버전이 다를 수 있는데, 가상환경을 사용하면 이런 충돌을 방지할 수 있습니다.

> 💡 ***왜 가상환경이 필요한가?*** *프로젝트 A는 `numpy 1.24` 를, 프로젝트 B는 `numpy 1.26` 을 필요로 한다면, 가상환경 없이는 하나의 버전만 설치할 수 있어 충돌이 발생합니다. 가상환경을 사용하면 각 프로젝트가 자신만의 독립된 패키지 공간을 갖게 됩니다.*

가상환경 생성

프로젝트 폴더를 만들고 그 안에서 가상환경을 생성합니다. Python 내장 모듈인 `venv` 를 사용합니다.

```
# 프로젝트 폴더 생성 및 이동
mkdir my-ai-project
cd my-ai-project

# 가상환경 생성 (.venv 폴더 이름은 자유롭게 지정 가능)
python -m venv .venv
```

가상환경 활성화

가상환경을 생성했다면 반드시 활성화해야 합니다. 활성화하면 터미널 프롬프트 앞에 `(.venv)` 가 표시됩니다.

```
# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# Windows (명령 프롬프트)
.venv\Scripts\activate.bat

# macOS / Linux
source .venv/bin/activate
```

활성화 후 터미널이 아래처럼 보이면 정상입니다.

```
(.venv) C:\my-ai-project>
```

가상환경 비활성화

작업이 끝나면 아래 명령어로 가상환경을 비활성화합니다.

```
deactivate
```

VS Code에서 가상환경 연결

VS Code에서 해당 프로젝트 폴더를 열면 자동으로 `.venv` 를 감지합니다. 감지하지 못하는 경우 `Ctrl+Shift+P` → `Python: Select Interpreter` 에서 `.venv/bin/python` (또는 `.venv\Scripts\python.exe`)을 직접 선택합니다.

> ⚠️ ***주의**: 터미널에서 가상환경을 활성화한 상태에서 패키지를 설치해야 해당 프로젝트에만 패키지가 설치됩니다. 가상환경을 활성화하지 않으면 시스템 전체에 설치될 수 있습니다.*

1.5 필수 라이브러리 설치

AI/데이터 분석을 위한 핵심 라이브러리 4가지를 설치합니다. 가상환경을 활성화한 상태에서 아래 명령어를 실행합니다.

```
pip install numpy pandas matplotlib scikit-learn
```

각 라이브러리의 역할을 간단히 살펴보겠습니다.

NumPy (넘파이)

NumPy(Numerical Python)는 수치 계산을 위한 핵심 라이브러리입니다. 다차원 배열(`ndarray`)을 제공하며, 행렬 연산, 통계 계산 등을 빠르게 처리합니다.

```
import numpy as np

# 1차원 배열 생성
arr = np.array([1, 2, 3, 4, 5])
print(arr)         # [1 2 3 4 5]
print(arr.mean())  # 3.0 (평균)
print(arr * 2)     # [2 4 6 8 10] (벡터 연산)
```

Pandas (판다스)

Pandas는 표 형태의 데이터(CSV, Excel 등)를 다루는 라이브러리입니다. `DataFrame`이라는 자료구조를 통해 데이터를 쉽게 불러오고, 정제하고, 분석할 수 있습니다.

```
import pandas as pd

# 간단한 데이터프레임 생성
data = {
    "이름": ["Alice", "Bob", "Charlie"],
    "나이": [25, 30, 35],
    "점수": [88, 92, 78]
}
df = pd.DataFrame(data)
print(df)
print(df["점수"].mean())  # 86.0 (점수 평균)
```

Matplotlib (맷플롯립)

Matplotlib은 데이터를 시각화하는 라이브러리입니다. 선 그래프, 막대 그래프, 산점도 등 다양한 차트를 그릴 수 있습니다.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y, marker='o')
plt.title("간단한 선 그래프")
plt.xlabel("X축")
plt.ylabel("Y축")
plt.show()
```

Scikit-learn (사이킷런)

Scikit-learn(sklearn)은 머신러닝 알고리즘을 쉽게 사용할 수 있도록 모아놓은 라이브러리입니다. 분류, 회귀, 군집화, 전처리 등 다양한 기능을 일관된 API로 제공합니다.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# 붓꽃 데이터셋 불러오기
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)
print(f"훈련 데이터: {X_train.shape}, 테스트 데이터: {X_test.shape}")
```

설치 확인

아래 코드로 모든 라이브러리가 정상적으로 설치되었는지 확인합니다.

```
import numpy as np
import pandas as pd
import matplotlib
import sklearn

print(f"NumPy 버전: {np.__version__}")
print(f"Pandas 버전: {pd.__version__}")
print(f"Matplotlib 버전: {matplotlib.__version__}")
print(f"Scikit-learn 버전: {sklearn.__version__}")
print("모든 라이브러리가 정상적으로 설치되었습니다!")
```

1.6 인공지능의 정의와 분류

인공지능(Artificial Intelligence, AI)은 인간의 지능적 행동(학습, 추론, 문제 해결, 언어 이해 등)을 컴퓨터 프로그램으로 구현하는 기술을 말합니다.

AI는 능력의 범위에 따라 크게 세 가지로 분류됩니다.

약한 AI (Narrow AI / Weak AI)

약한 AI(Narrow AI)는 특정 작업 하나에만 뛰어난 AI를 말합니다. 현재 우리가 실생활에서 사용하는 AI는 거의 모두 약한 AI에 해당합니다.

- 바둑을 두는 AlphaGo (바둑 외의 다른 게임은 못함)

- 이미지를 인식하는 AI (이미지 인식 외의 작업은 못함)
- ChatGPT와 같은 언어 모델 (텍스트 처리 전문)
- 음성을 인식하는 Siri, Alexa, 클로바

강한 AI (General AI / Strong AI)

강한 AI(AGI, Artificial General Intelligence)는 인간처럼 어떤 지적 과제도 처리할 수 있는 AI를 말합니다. 한 분야를 학습하면 다른 분야에도 그 지식을 활용할 수 있는 범용 지능입니다.

> ⚠️ ***현실**: 2026년 현재 강한 AI는 아직 존재하지 않습니다. 이는 AI 연구자들의 궁극적인 목표 중 하나입니다.*

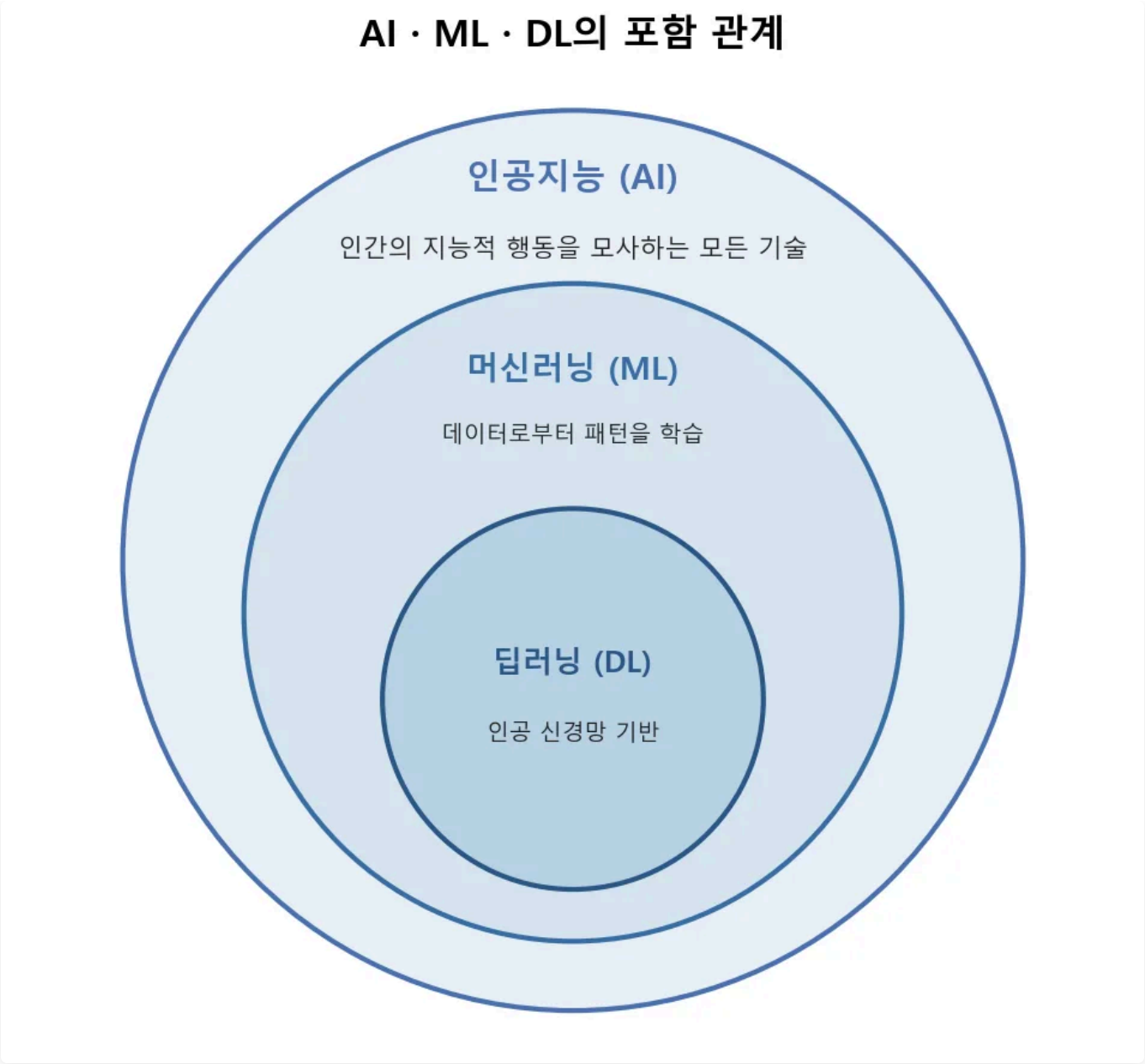
초지능 (Superintelligence)

초지능(Superintelligence)은 모든 분야에서 인간의 능력을 월등히 초월하는 가상의 AI를 말합니다. 철학자 닉 보스트롬(Nick Bostrom)이 자신의 저서에서 체계적으로 논의한 개념입니다.

> 💡 ***정리**: 현재 AI 기술의 수준은 **약한 AI** 단계에 있습니다. 하지만 약한 AI도 특정 분야에서는 이미 인간을 능가하는 성능을 보여주고 있습니다.*

AI, 머신러닝, 딥러닝의 관계

많은 분들이 AI, **머신러닝(Machine Learning, ML)**, **딥러닝(Deep Learning, DL)**을 혼용하는데, 이 셋은 포함 관계에 있습니다.



- **AI**: 가장 넓은 개념. 인간의 지능을 흉내 내는 모든 기술
- **머신러닝**: AI의 한 분야. 규칙을 직접 프로그래밍하는 대신 데이터로부터 자동으로 학습
- **딥러닝**: 머신러닝의 한 분야. 인공 신경망을 여러 층 쌓아 복잡한 패턴을 학습

1.7 AI가 할 수 있는 것과 할 수 없는 것

AI 기술에 대한 과도한 기대와 불필요한 두려움을 줄이기 위해, AI가 실제로 무엇을 잘하고 무엇이 어려운지 명확히 이해하는 것이 중요합니다.

AI가 잘하는 것

- **패턴 인식**: 수많은 데이터에서 규칙이나 패턴을 찾아냅니다. 이미지 분류, 스팸 메일 필터링 등이 대표적입니다.
- **반복 작업 자동화**: 동일한 유형의 작업을 빠르고 일관되게 처리합니다. 24시간 쉬지 않고 작동할 수 있습니다.
- **대용량 데이터 처리**: 인간이 평생 읽어도 다 못 읽을 양의 데이터를 짧은 시간에 분석합니다.
- **특정 게임/최적화**: 바둑, 체스, 특정 비디오 게임에서 인간을 압도하는 성능을 보입니다.
- **언어 생성 및 번역**: 자연스러운 텍스트를 생성하고 여러 언어 간 번역을 수행합니다.

AI가 어려워하는 것

- **상식적 추론**: "유리컵을 바닥에 떨어뜨리면 어떻게 될까?"처럼 인간에게는 당연한 상식을 AI는 데이터 없이 추론하기 어렵습니다.
- **인과관계 이해**: 상관관계는 잘 찾아내지만, "왜 그런지"를 진정으로 이해하는 것은 어렵습니다.
- **적은 데이터로 학습**: AI는 일반적으로 대용량 학습 데이터가 필요합니다. 인간 아이는 강아지를 몇 번만 봐도 구분하지만, AI는 수천 장의 사진이 필요할 수 있습니다.
- **감정과 공감**: 텍스트로 감정을 표현할 수 있지만, 실제로 감정을 느끼거나 진정한 공감을 하는 것은 불가능합니다.
- **완전히 새로운 창의성**: 기존 데이터의 패턴을 조합하는 것은 잘하지만, 완전히 새로운 아이디어를 만들어내는 창의성에는 한계가 있습니다.

> 📌 ***핵심**: AI는 도구입니다. 망치로 나사를 조이려 하면 안 되듯, AI도 잘 맞는 문제에 적용했을 때 최고의 성능을 발휘합니다. AI의 강점과 한계를 정확히 이해하는 것이 좋은 AI 활용자가 되는 첫걸음입니다.*

1.8 인공지능의 발전 역사

AI의 역사를 이해하면 현재의 AI 기술이 왜 이런 형태로 발전했는지 맥락을 파악할 수 있습니다.

1950년대: AI의 시작 - 튜링 테스트

앨런 튜링(Alan Turing)은 1950년 "기계가 생각할 수 있는가?"라는 질문을 던지며 **튜링 테스트(Turing Test)**를 제안했습니다. 이 테스트는 인간이 대화 상대가 사람인지 기계인지 구분하지 못하면 그 기계는 "지능이 있다"고 볼 수 있다는 개념입니다.

같은 시기인 1956년, **존 매카시(John McCarthy)**가 "Artificial Intelligence"라는 용어를 처음 사용했으며, 이를 AI 연구의 공식적인 시작으로 봅니다.

1980~90년대: AI의 겨울과 전문가 시스템

AI는 초기의 기대와 달리 한계에 부딪히며 두 차례의 **AI 겨울(AI Winter)** 시기를 겪었습니다. 연구비 지원이 끊기고 관심이 급감한 기간입니다.

이 시기에 **전문가 시스템(Expert System)**이 등장했습니다. 전문가의 지식을 규칙으로 프로그래밍하여 특정 분야의 문제를 해결하는 방식이었지만, 새로운 상황에 유연하게 대응하지 못하는 한계가 있었습니다.

1997년: 딥블루 vs 카스파로프

IBM이 개발한 체스 AI **딥블루(Deep Blue)**가 당시 세계 체스 챔피언 **가리 카스파로프(Garry Kasparov)**를 꺾었습니다. AI가 특정 지적 게임에서 인간 최고 전문가를 처음으로 이긴 역사적인 사건입니다.

2012년: 딥러닝의 부활 - ImageNet 대회

알렉스넷(AlexNet)이 ImageNet 이미지 인식 대회에서 기존 방식을 압도하는 성능으로 우승했습니다. 이 사건은 딥러닝 붐의 시작점으로, GPU를 활용한 대규모 신경망 학습의 가능성을 증명했습니다.

2016년: 알파고 vs 이세돌

구글 딥마인드의 **알파고(AlphaGo)**가 바둑 세계 챔피언 **이세돌** 9단을 4:1로 꺾었습니다. 바둑은 경우의 수가 우주의 원자 수보다 많아 인간을 절대 이기지 못할 것으로 여겨졌기 때문에, 이 사건은 전 세계에 AI의 가능성을 강렬하게 알렸습니다.

2017년: 트랜스포머 아키텍처 등장

구글의 연구팀이 "Attention Is All You Need" 논문을 통해 **트랜스포머(Transformer)** 아키텍처를 발표했습니다. 이 구조는 이후 모든 대형 언어 모델(LLM)의 기반이 되었습니다.

2022년: ChatGPT의 등장

OpenAI가 출시한 **ChatGPT**는 공개 5일 만에 100만 명의 사용자를 확보하며 폭발적인 반응을 불러일으켰습니다. 복잡한 질문에 자연스럽게 대답하고, 코드를 작성하고, 글을 써주는 능력은 AI의 대중화 시대를 열었습니다.

> 💡 ***현재**: 2026년 현재 **생성형 AI(Generative AI)**는 텍스트, 이미지, 음악, 영상 생성 등 다양한 분야로 빠르게 확장되고 있으며, 산업 전반에 걸쳐 혁신을 주도하고 있습니다.*

AI 발전 역사를 타임라인으로 정리하면 다음과 같습니다.



시기	주요 사건
1950	튜링 테스트 제안 (앨런 튜링)
1956	"Artificial Intelligence" 용어 탄생 (존 매카시)
1970s	1차 AI 겨울
1980s	전문가 시스템 전성기
1990s	2차 AI 겨울
1997	딥블루, 체스 챔피언 카스파로프 격파
2012	AlexNet, ImageNet 대회 압도적 우승 (딥러닝 부활)
2016	알파고, 이세돌 9단 4:1 격파
2017	트랜스포머 아키텍처 등장
2020	GPT-3 공개
2022	ChatGPT 출시 → AI 대중화 시대
2026	생성형 AI의 일상적 활용

[실습] Python 기본 문법 확인 및 첫 프로그램 실행

이제 직접 Python 코드를 작성하고 실행해보겠습니다. VS Code를 열고 가상환경을 활성화한 상태에서 진행합니다.

실습 환경 준비

1. VS Code에서 프로젝트 폴더를 열고, 가상환경(`.venv`)이 활성화되어 있는지 확인합니다.
2. `chapter01` 폴더를 만들고 `practice.py` 파일을 생성합니다.

실습 1: Python 기본 문법 확인

아래 코드를 `practice.py` 에 작성하고 실행합니다. `F5` 키 또는 우측 상단의 실행 버튼을 누릅니다.

```python
# ============================================================
# Chapter 1 실습: Python 기본 문법 확인
# ============================================================

# 1. 변수와 자료형
name = "Alice"          # 문자열 (str)
age = 25                # 정수 (int)
score = 92.5            # 실수 (float)
is_student = True       # 불리언 (bool)

print("=== 변수와 자료형 ===")
print(f"이름: {name}, 나이: {age}, 점수: {score}, 학생: {is_student}")
print(f"name의 자료형: {type(name)}")

# 2. 리스트와 반복문
print("\n=== 리스트와 반복문 ===")
fruits = ["사과", "바나나", "딸기", "포도"]
for i, fruit in enumerate(fruits):
    print(f"{i+1}번째 과일: {fruit}")

# 3. 조건문
print("\n=== 조건문 ===")
temperature = 28
if temperature >= 30:
    print("더운 날씨입니다.")
elif temperature >= 20:
    print("쾌적한 날씨입니다.")
else:
    print("쌀쌀한 날씨입니다.")

# 4. 함수 정의
print("\n=== 함수 ===")
def greet(name, language="한국어"):
    if language == "한국어":
        return f"안녕하세요, {name}님!"
    else:
        return f"Hello, {name}!"

print(greet("Bob"))
print(greet("Alice", language="영어"))

# 5. 딕셔너리
```

```
print("\n=== 딕셔너리 ===")
student = {
    "name": "Charlie",
    "age": 22,
    "scores": [85, 90, 78, 95]
}
avg_score = sum(student["scores"]) / len(student["scores"])
print(f"학생: {student['name']}, 평균 점수: {avg_score:.1f}")
```

실습 2: 라이브러리 설치 확인 및 간단한 데이터 시각화

아래 코드로 설치한 라이브러리를 실제로 사용해봅니다.

```
# ============================================================
# Chapter 1 실습: 라이브러리 활용
# ============================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# --- NumPy: 수치 계산 ---
print("═══ NumPy 실습 ===")
scores = np.array([85, 92, 78, 96, 88, 73, 91, 84])
print(f"점수 데이터: {scores}")
print(f"평균: {scores.mean():.1f}")
print(f"최고점: {scores.max()}, 최저점: {scores.min()}")
print(f"표준편차: {scores.std():.2f}")

# --- Pandas: 데이터프레임 ---
print("\n═══ Pandas 실습 ===")
data = {
    "학생": ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "수학": [85, 92, 78, 96, 88],
    "영어": [90, 85, 82, 89, 94],
    "과학": [88, 79, 91, 87, 76]
}
df = pd.DataFrame(data)
df["평균"] = df[["수학", "영어", "과학"]].mean(axis=1).round(1)
print(df.to_string(index=False))
print(f"\n과목별 평균:\n{df[['수학', '영어', '과학']].mean().round(1)}")

# --- Matplotlib: 시각화 ---
print("\n═══ Matplotlib 실습 (차트 창이 열립니다) ===")
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# 막대 그래프: 학생별 평균 점수
axes[0].bar(df["학생"], df["평균"], color=["#4C72B0", "#55A868", "#C44E52", "#8172B2", "#CCB97
4"])
axes[0].set_title("학생별 평균 점수")
axes[0].set_xlabel("학생")
axes[0].set_ylabel("점수")
axes[0].set_ylim(0, 100)

# 꺾은선 그래프: 과목별 점수 비교
subjects = ["수학", "영어", "과학"]
```

```
for _, row in df.iterrows():
    axes[1].plot(subjects, [row["수학"], row["영어"], row["과학"]], marker='o', label=row["학생"])
axes[1].set_title("과목별 점수 비교")
axes[1].set_xlabel("과목")
axes[1].set_ylabel("점수")
axes[1].legend()
axes[1].set_ylim(60, 100)

plt.tight_layout()
plt.show()
print("실습 완료!")
```

예상 출력 결과

코드를 정상적으로 실행하면 터미널에 아래와 비슷한 결과가 출력되고, 차트 창이 열립니다.

```
=== NumPy 실습 ===
점수 데이터: [85 92 78 96 88 73 91 84]
평균: 85.9
최고점: 96, 최저점: 73
표준편차: 6.87

=== Pandas 실습 ===
  학생  수학  영어  과학   평균
 Alice  85  90  88  87.7
   Bob  92  85  79  85.3
...
```

> 💡 ***오류가 발생하면?*** *`ModuleNotFoundError: No module named 'numpy'` 와 같은 오류가 발생하면 가상환경이 활성화되지 않은 상태에서 실행한 것입니다. 가상환경을 활성화하고 `pip install numpy pandas matplotlib` 명령어를 다시 실행해보세요.*

마치며

이번 챕터에서 학습한 내용을 정리합니다.

- **개발환경 구축**: Python 3.11 설치, VS Code 설정, 가상환경(venv) 생성과 관리 방법을 익혔습니다.
- **핵심 라이브러리**: NumPy(수치계산), Pandas(데이터처리), Matplotlib(시각화), Scikit-learn(머신러닝) 4가지 핵심 라이브러리를 설치하고 기본 사용법을 확인했습니다.
- **AI의 정의와 분류**: 약한 AI / 강한 AI / 초지능의 차이, 그리고 AI-머신러닝-딥러닝의 포함 관계를 이해했습니다.
- **AI의 한계**: AI는 강력한 도구이지만 상식적 추론, 진정한 창의성, 감정 등에는 한계가 있음을 배웠습니다.
- **AI 발전 역사**: 튜링 테스트(1950) → 딥블루(1997) → AlexNet(2012) → 알파고(2016) → ChatGPT(2022)로 이어지는 AI의 역사적 흐름을 파악했습니다.

> 📌 ***다음 챕터 예고**: Chapter 2에서는 데이터 분석의 기초를 다룹니다. NumPy와 Pandas를 본격적으로 활용하여 실제 데이터를 불러오고, 정제하고, 탐색하는 방법을 배웁니다. 이번 챕터에서 구축한 환경이 앞으로 모든 실습의 토대가 됩니다.*

지도학습은 머신러닝에서 가장 널리 사용되는 학습 방식입니다. 정답이 있는 데이터를 통해 컴퓨터가 스스로 규칙을 학습하도록 만드는 방법으로, 이메일 스팸 필터부터 의료 진단 보조까지 실생활의 수많은 곳에 활용되고 있습니다.

이 챕터에서는 지도학습의 핵심 개념을 차근차근 쌓아 올리고, 직접 Python 코드로 분류 모델을 만들어 보겠습니다.

> 📌 ***학습 목표*** *- 규칙 기반 접근법과 데이터 기반 접근법(머신러닝)의 차이를 이해합니다. - 학습 데이터, 레이블, 특성(Feature)의 개념을 설명할 수 있습니다. - 분류와 회귀 문제의 차이를 파악하고, 대표 알고리즘을 익힙니다. - 모델 평가 지표를 이해하고 상황에 맞는 지표를 선택할 수 있습니다. - 과적합·과소적합의 원인과 해결 방법을 설명할 수 있습니다. - scikit-learn으로 간단한 분류 모델을 직접 구현합니다.*

2.1 머신러닝 개요 — 규칙 기반 vs 데이터 기반

규칙 기반 접근법

컴퓨터에게 어떤 일을 시키는 가장 전통적인 방법은 **규칙(rule)을 직접 작성**하는 것입니다. 예를 들어 "제목에 '광고'나 '무료'라는 단어가 들어 있으면 스팸 메일로 분류하라"는 식입니다.

규칙 기반 방식은 직관적이지만, 현실은 훨씬 복잡합니다. 스팸 발송자는 곧 '광고' 대신 '광ㅇ고'처럼 변형된 단어를 사용하기 시작하고, 개발자는 새 규칙을 계속 추가해야 합니다. 규칙이 수백 개를 넘어서면 유지·보수가 사실상 불가능해집니다.

데이터 기반 접근법 (머신러닝)

머신러닝(Machine Learning)은 사람이 규칙을 직접 작성하는 대신, **데이터로부터 컴퓨터가 스스로 규칙을 학습**하도록 하는 접근법입니다.

"스팸이다 / 아니다"라고 레이블된 수천 개의 이메일을 제공하면, 머신러닝 알고리즘은 그 안에서 패턴을 찾아내어 새로운 이메일을 판별하는 규칙을 자동으로 만들어 냅니다.

아래 표는 두 접근법의 차이를 정리한 것입니다.

구분	규칙 기반	데이터 기반 (머신러닝)
규칙 작성 주체	사람 (개발자)	알고리즘 (데이터에서 자동 학습)
필요한 것	도메인 전문 지식	충분한 양의 데이터
유연성	낮음 (규칙 변경 시 재작성)	높음 (데이터 추가로 성능 개선)
복잡한 패턴 처리	어려움	강함
해석 가능성	높음	알고리즘에 따라 다름

> 💡 *머신러닝이 모든 경우에 최선은 아닙니다. 데이터가 적거나 명확한 규칙이 존재할 때는 규칙 기반이 더 빠르고 신뢰할 수 있습니다.*

2.2 학습 데이터, 레이블, 특성(Feature)의 개념

머신러닝 모델을 만들기 전에 반드시 이해해야 할 세 가지 핵심 개념이 있습니다.

학습 데이터 (Training Data)

학습 데이터(Training Data)는 모델이 규칙을 익히기 위해 사용하는 데이터 모음입니다. 학교에서 교과서로 공부하는 것처럼, 모델은 학습 데이터를 반복적으로 보며 패턴을 파악합니다.

레이블 (Label)

레이블(Label)은 각 데이터의 정답입니다. 지도학습에서 '지도(supervised)'라는 말이 붙은 이유가 바로 이 레이블 때문입니다. 선생님이 학생에게 정답을 알려주듯, 알고리즘에게 각 데이터가 어떤 결과를 가져야 하는지 알려 줍니다.

- 분류 문제의 레이블 예시: "스팸" / "정상", "고양이" / "강아지" / "새"
- 회귀 문제의 레이블 예시: 집 가격(숫자), 내일 기온(숫자)

특성 (Feature)

특성(Feature)은 모델이 예측에 활용하는 입력 변수입니다. 집 가격을 예측한다면 면적, 방 개수, 위치 등이 특성이 됩니다. 좋은 특성을 선택하고 가공하는 작업을 **특성 공학(Feature Engineering)**이라고 하며, 모델 성능에 큰 영향을 줍니다.

> 📌 *데이터를 하나의 표(테이블)로 상상해 보세요. **행(row)**은 각 데이터 샘플이고, **열(column)**은 특성입니다. 마지막 열에 정답인 레이블이 붙어 있는 형태가 전형적인 지도학습 데이터셋입니다.*

아래는 개념을 정리한 표입니다.

용어	다른 표현	설명
학습 데이터	Training Set	모델 학습에 사용하는 데이터
레이블	Target, 정답, y	예측해야 할 값
특성	Feature, 입력 변수, X	예측에 사용하는 입력 값
샘플	Instance, 관측치	데이터 하나의 행

2.3 분류(Classification) 문제

분류(Classification)는 입력 데이터가 어떤 **범주(category)**에 속하는지 예측하는 문제입니다. 출력값이 이산적(discrete)이라는 것이 핵심입니다.

- 이진 분류(Binary Classification): 두 가지 범주 (스팸/정상, 양성/음성)
- 다중 분류(Multi-class Classification): 세 가지 이상 범주 (꽃 품종 3가지, 숫자 0~9)

의사결정트리 (Decision Tree)

의사결정트리(Decision Tree)는 "키가 170 이상인가? → Yes면 오른쪽, No면 왼쪽"처럼 질문을 반복하여 데이터를 분류하는 알고리즘입니다. 결과를 나무 구조로 시각화할 수 있어 해석이 쉽습니다.

> 💡 *의사결정트리는 직관적이고 해석 가능하다는 장점이 있지만, 학습 데이터에 과적합되기 쉬운 단점이 있습니다.*

K-최근접 이웃 (K-Nearest Neighbors, KNN)

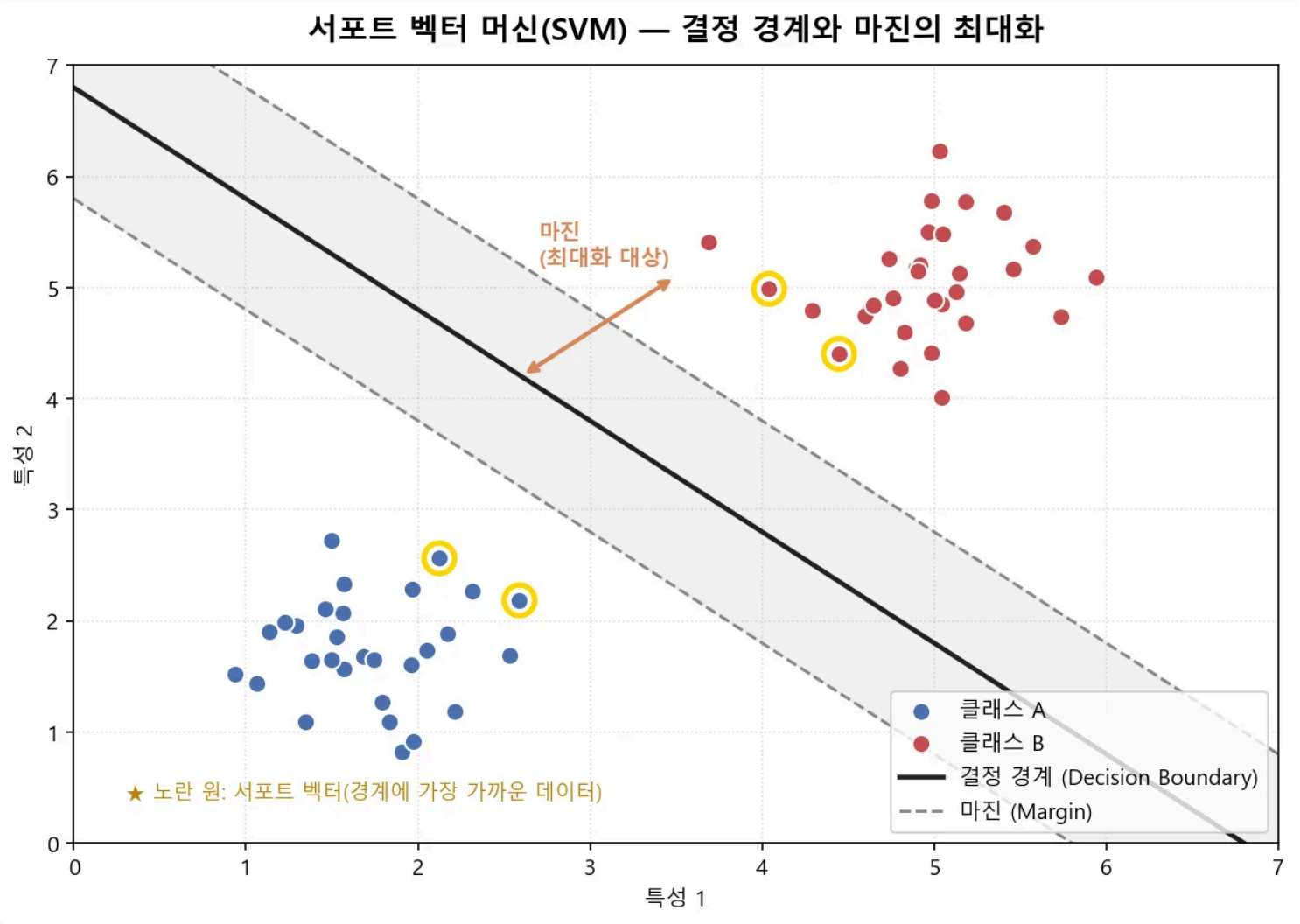
K-최근접 이웃(KNN)은 새로운 데이터가 들어오면, 기존 학습 데이터 중 가장 가까운 K개의 이웃을 찾아 다수결로 분류하는 알고리즘입니다. 거리 계산에는 주로 **유클리드 거리(Euclidean Distance)**를 사용합니다.

$$d(p, q) = \sqrt{\sum_{i=1}^{n}(p_i - q_i)^2}$$

K 값이 너무 작으면 노이즈에 민감해지고, 너무 크면 경계가 흐릿해집니다. 적절한 K 값을 선택하는 것이 중요합니다.

서포트 벡터 머신 (Support Vector Machine, SVM)

서포트 벡터 머신(SVM)은 두 클래스 사이의 경계선(결정 경계, Decision Boundary)을 찾는 알고리즘입니다. 단순히 경계를 긋는 것이 아니라, 양쪽 클래스와의 거리(마진, Margin)가 **최대**가 되는 경계를 찾습니다.



경계에 가장 가까운 데이터(노란 원)를 **서포트 벡터**라고 부르며, 결정 경계는 오직 이 점들에 의해 결정됩니다.

> 📌 *SVM은 고차원 데이터에서도 잘 작동하며, 커널 함수(Kernel Function)를 이용해 비선형 경계도 처리할 수 있습니다.*

아래 표는 세 알고리즘을 비교한 것입니다.

알고리즘	핵심 아이디어	장점	단점
의사결정트리	질문(조건) 분기	해석 용이, 빠름	과적합 위험
KNN	가장 가까운 K개 이웃 다수결	단순, 직관적	느림, 메모리 많이 사용
SVM	최대 마진 결정 경계	고차원 효과적, 강인	대용량 데이터에 느림

2.4 회귀(Regression) 문제

회귀(Regression)는 입력 데이터를 받아 **연속적인 숫자값**을 예측하는 문제입니다. 분류가 "어떤 종류인가?"를 묻는다면, 회귀는 "얼마인가?"를 묻습니다.

- 예시: 집 가격 예측, 내일 기온 예측, 판매량 예측

선형 회귀 (Linear Regression)

선형 회귀(Linear Regression)는 입력 특성과 출력값 사이의 관계를 직선(1차 함수)으로 모델링합니다.

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

여기서 $\hat{y}$는 예측값, w_0는 편향(bias), $w_1, \ldots, w_n$은 각 특성에 대한 가중치(weight)입니다. 학습 과정에서는 예측값과 실제값의 차이(오차)를 최소화하는 방향으로 가중치를 조정합니다. 이 오차를 측정하는 대표적인 함수가 **평균 제곱 오차(MSE, Mean Squared Error)**입니다.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

다항 회귀 (Polynomial Regression)

실제 데이터는 직선이 아닌 곡선 형태의 관계를 가질 수 있습니다. **다항 회귀(Polynomial Regression)**는 특성에 제곱, 세제곱 등의 다항식 항을 추가하여 곡선 관계를 모델링합니다.

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \cdots$$

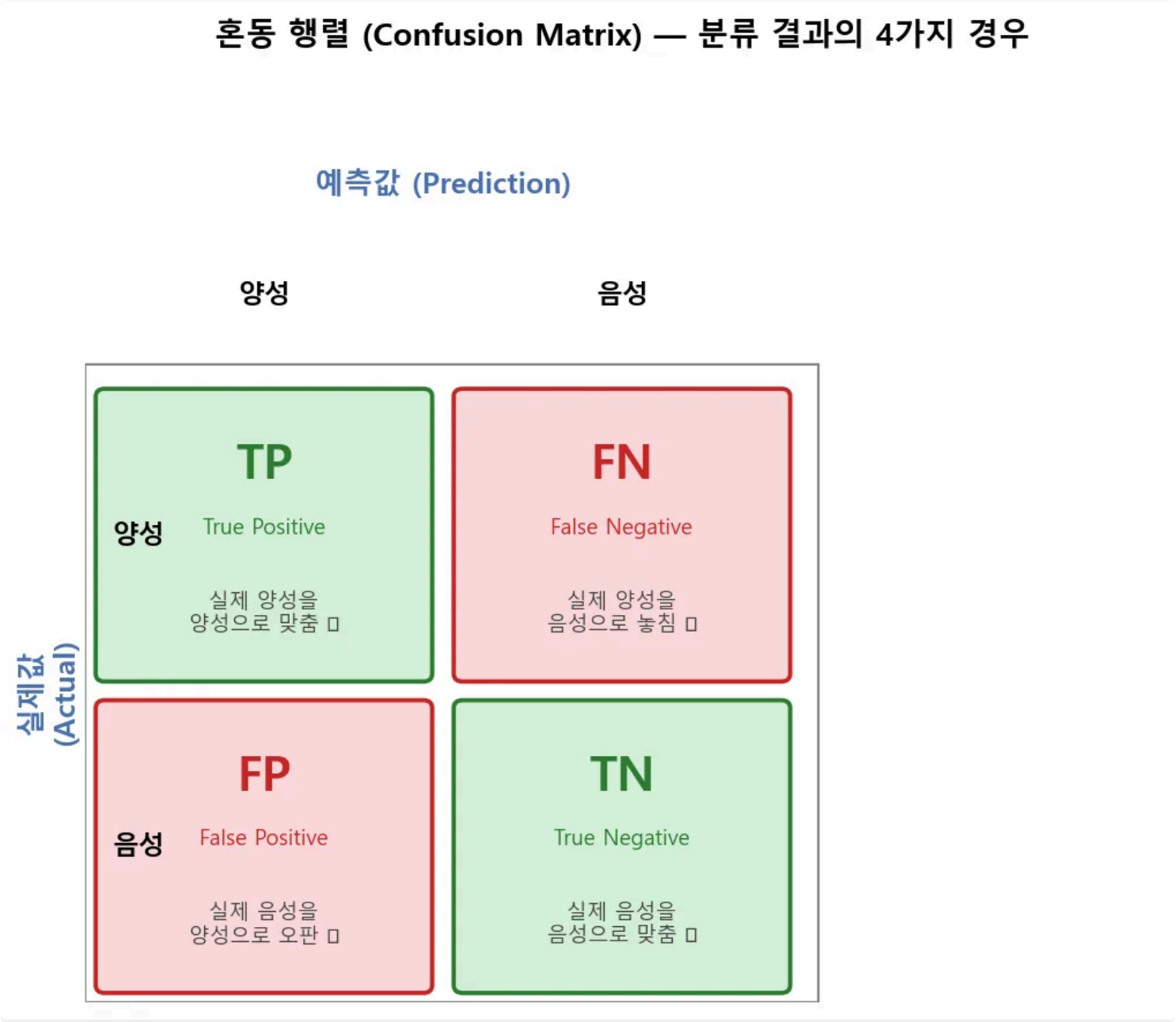
> ⚠️ *다항식의 차수가 너무 높아지면 학습 데이터에 과적합될 위험이 커집니다. 적절한 차수 선택이 중요합니다.*

2.5 모델 평가 지표

모델을 만든 후에는 성능을 수치로 측정해야 합니다. 문제 유형(분류/회귀)에 따라 사용하는 지표가 다릅니다.

분류 평가 지표

분류 문제의 예측 결과는 아래 네 가지로 나뉩니다. 이를 **혼동 행렬(Confusion Matrix)**이라고 합니다.



	예측: 양성(Positive)	예측: 음성(Negative)
실제: 양성	TP (True Positive)	FN (False Negative)
실제: 음성	FP (False Positive)	TN (True Negative)

정확도(Accuracy)는 전체 중 올바르게 예측한 비율입니다.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

정밀도(Precision)는 양성으로 예측한 것 중 실제로 양성인 비율입니다. "양성이라고 외쳤을 때 얼마나 맞았는가?"

$$\text{Precision} = \frac{TP}{TP + FP}$$

재현율(Recall)은 실제 양성 중 양성으로 예측한 비율입니다. "실제 양성을 얼마나 잘 찾아냈는가?"

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score는 정밀도와 재현율의 조화 평균으로, 두 지표의 균형을 나타냅니다.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

> 💡 *정밀도와 재현율은 트레이드오프 관계입니다. 암 진단처럼 놓치면 안 되는 상황(재현율 중요) vs. 스팸 필터처럼 정상 메일을 스팸으로 잘못 분류하면 안 되는 상황(정밀도 중요) 에 따라 우선순위가 달라집니다.*

회귀 평가 지표

RMSE(Root Mean Squared Error)는 예측 오차의 크기를 나타내는 대표 지표입니다. 단위가 원래 값과 같아 해석이 직관적입니다.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

아래 표는 주요 평가 지표를 정리한 것입니다.

지표	문제 유형	공식	설명
Accuracy	분류	$(TP+TN)/전체$	전체 정확도
Precision	분류	$TP/(TP+FP)$	예측 양성의 정확성
Recall	분류	$TP/(TP+FN)$	실제 양성 탐지율
F1-Score	분류	정밀도·재현율 조화평균	균형 지표
RMSE	회귀	$\sqrt{MSE}$	오차 크기

2.6 학습/검증/테스트 데이터 분할

모델을 제대로 평가하려면 데이터를 세 부분으로 나누어야 합니다.

- **학습 세트(Training Set)**: 모델이 규칙을 학습하는 데이터 (보통 60~70%)
- **검증 세트(Validation Set)**: 하이퍼파라미터 튜닝 및 모델 선택에 사용 (보통 10~20%)
- **테스트 세트(Test Set)**: 최종 성능 평가에 딱 한 번만 사용 (보통 10~20%)

> ⚠️ *테스트 세트는 모델 개발 과정에서 절대 들여다보면 안 됩니다. 마치 시험 당일에 처음 보는 문제처럼, 완성된 모델의 실제 실력을 측정하는 데만 사용해야 합니다.*

데이터가 충분하지 않을 때는 **교차 검증(Cross-Validation)**을 사용합니다. 대표적인 **K-겹 교차 검증(K-Fold Cross-Validation)**은 데이터를 K개의 fold로 나누어, 각 fold를 번갈아 검증 세트로 사용합니다.

scikit-learn으로 데이터를 분할하는 코드는 다음과 같습니다.

```
from sklearn.model_selection import train_test_split

# 학습(70%)과 나머지(30%) 분할
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 나머지를 검증(50%)과 테스트(50%)로 분할 → 전체의 15%, 15%
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
```

2.7 과적합(Overfitting)과 과소적합(Underfitting)

모델 학습에서 가장 중요한 개념 중 하나가 **과적합(Overfitting)**과 **과소적합(Underfitting)**입니다.

과소적합 (Underfitting)

과소적합은 모델이 너무 단순하여 학습 데이터의 패턴조차 제대로 학습하지 못한 상태입니다. 학습 데이터와 테스트 데이터 모두 성능이 낮습니다.

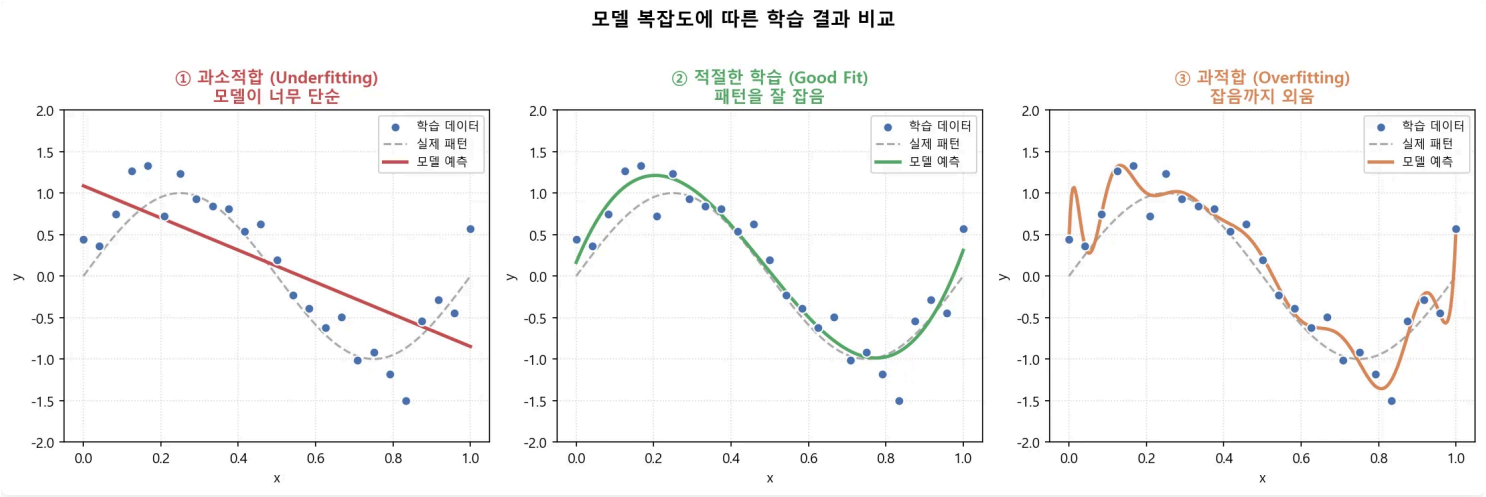
- 원인: 모델이 너무 단순, 학습 횟수 부족, 특성이 너무 적음
- 해결: 더 복잡한 모델 사용, 특성 추가, 학습 반복 횟수 증가

과적합 (Overfitting)

과적합은 모델이 학습 데이터를 너무 완벽하게 외워버린 나머지, 새로운 데이터(테스트 데이터)에서는 성능이 크게 떨어지는 상태입니다.

- 원인: 모델이 너무 복잡, 학습 데이터가 너무 적음, 학습 횟수 과도
- 해결: 더 많은 학습 데이터, 정규화(Regularization), 드롭아웃(Dropout), 조기 종료(Early Stopping)

아래 그림과 표는 세 가지 상태를 비교합니다.



상태	학습 성능	테스트 성능	설명
과소적합	낮음	낮음	패턴 학습 자체가 부족
적절한 학습	높음	높음	이상적인 상태
과적합	매우 높음	낮음	학습 데이터를 암기

> 💡 ***편향-분산 트레이드오프(Bias-Variance Tradeoff)****: 과소적합은 높은 편향(bias)과 관련이 있고, 과적합은 높은 분산(variance)과 관련이 있습니다. 좋은 모델은 이 둘 사이의 균형을 찾습니다.*

[실습] scikit-learn으로 분류 모델 만들기 (Iris 데이터셋)

이제 배운 개념을 코드로 직접 구현해 보겠습니다. 머신러닝 입문에서 가장 유명한 **Iris(붓꽃) 데이터셋**을 사용합니다. 꽃잎·꽃받침의 길이/너비 4가지 특성으로 3가지 품종을 분류합니다.

1단계: 라이브러리 및 데이터 불러오기

아래 코드로 필요한 라이브러리와 데이터를 준비합니다.

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# 데이터 로드
iris = load_iris()
X = iris.data        # 특성: (150, 4)
y = iris.target      # 레이블: 0, 1, 2

# DataFrame으로 확인
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = y
print(df.head())
print(f"\n데이터 크기: {X.shape}")
print(f"클래스 이름: {iris.target_names}")
```

2단계: 데이터 분할 및 전처리

데이터를 학습·테스트 세트로 나누고, 특성 값의 스케일을 맞춥니다.

```
# 학습(80%) / 테스트(20%) 분할
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 특성 스케일 정규화 (평균=0, 표준편차=1)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # 학습 세트로 fit + transform
X_test_scaled  = scaler.transform(X_test)        # 테스트 세트는 transform만

print(f"학습 데이터: {X_train_scaled.shape}")
print(f"테스트 데이터: {X_test_scaled.shape}")
```

> ⚠️ *스케일러는 반드시 학습 데이터만으로 `fit` 해야 합니다. 테스트 데이터의 정보가 학습에 새어 들어가는 **데이터 누수(Data Leakage)**를 막기 위함입니다.*

3단계: 세 가지 분류 모델 학습 및 비교

의사결정트리, KNN, SVM 세 모델을 학습하고 성능을 비교합니다.

```
models = {
    "의사결정트리": DecisionTreeClassifier(max_depth=4, random_state=42),
    "KNN (k=5)":   KNeighborsClassifier(n_neighbors=5),
    "SVM":         SVC(kernel='rbf', C=1.0, random_state=42),
}

results = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)          # 학습
    y_pred = model.predict(X_test_scaled)        # 예측
    acc = accuracy_score(y_test, y_pred)         # 정확도
    results[name] = acc
    print(f"\n[{name}] 정확도: {acc:.4f}")
    print(classification_report(y_test, y_pred,
                                target_names=iris.target_names))
```

4단계: 결과 시각화

각 모델의 정확도를 막대 그래프로 시각화합니다.

```
import matplotlib.pyplot as plt
import matplotlib

matplotlib.rcParams['font.family'] = 'Malgun Gothic'  # 한글 폰트 설정
matplotlib.rcParams['axes.unicode_minus'] = False

model_names = list(results.keys())
accuracies  = list(results.values())

plt.figure(figsize=(8, 5))
bars = plt.bar(model_names, accuracies, color=['steelblue', 'darkorange', 'seagreen'])
plt.ylim(0.8, 1.05)
plt.ylabel('정확도 (Accuracy)')
plt.title('Iris 데이터셋 분류 모델 성능 비교')

for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + 0.005,
             f'{acc:.4f}',
             ha='center', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()
```

5단계: 새로운 데이터 예측

학습된 모델로 새로운 꽃의 품종을 예측합니다.

```
# 새로운 꽃 데이터: [꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비]
new_flower = np.array([[5.1, 3.5, 1.4, 0.2]])

# 동일한 스케일러로 변환
new_flower_scaled = scaler.transform(new_flower)

# SVM 모델로 예측
svm_model = models["SVM"]
prediction = svm_model.predict(new_flower_scaled)
print(f"예측 품종: {iris.target_names[prediction[0]]}")
```

> 📌 ***실습 결과 해석**: 세 모델 모두 Iris 데이터셋에서 90% 이상의 높은 정확도를 보입니다. Iris는 비교적 분류가 쉬운 데이터셋이므로, 실제 문제에서는 이보다 낮은 수치가 나오는 것이 일반적입니다.*

마치며

이번 챕터에서는 지도학습의 핵심 개념을 모두 살펴보았습니다.

- **규칙 기반 vs 데이터 기반**: 머신러닝은 데이터로부터 규칙을 자동으로 학습합니다.
- **특성, 레이블, 학습 데이터**: 지도학습의 세 가지 핵심 재료입니다.
- **분류 알고리즘**: 의사결정트리, KNN, SVM은 각기 다른 방식으로 경계를 학습합니다.
- **회귀 알고리즘**: 선형 회귀와 다항 회귀로 연속적인 값을 예측합니다.
- **평가 지표**: 정확도, 정밀도, 재현율, F1-Score, RMSE는 목적에 맞게 선택해야 합니다.
- **데이터 분할**: 학습/검증/테스트 세트를 분리해야 공정한 평가가 가능합니다.
- **과적합과 과소적합**: 모델 복잡도와 데이터 양의 균형이 중요합니다.

> 📌 *다음 챕터에서는 레이블 없이 데이터의 구조를 스스로 파악하는 **비지도학습(Unsupervised Learning)**을 학습합니다. 군집화(Clustering)와 차원 축소(Dimensionality Reduction)를 중심으로 살펴보겠습니다.*

지금까지 우리는 정답(레이블)이 있는 데이터로 모델을 학습시키는 지도학습을 살펴보았습니다. 하지만 현실 세계의 데이터 대부분은 레이블이 없습니다. 이 챕터에서는 레이블 없이도 데이터 속 패턴을 스스로 찾아내는 **비지도학습(Unsupervised Learning)**과, 시행착오를 통해 스스로 최선의 행동을 학습하는 **강화학습(Reinforcement Learning)**을 소개합니다.

학습 목표

- 비지도학습의 개념과 지도학습과의 차이를 설명할 수 있다.
- K-Means, DBSCAN, 계층적 클러스터링의 원리와 차이를 이해한다.
- PCA를 통한 차원 축소의 개념을 이해한다.
- 이상탐지와 연관규칙의 실제 활용 사례를 설명할 수 있다.
- 강화학습의 핵심 구성요소(에이전트, 환경, 상태, 행동, 보상)를 설명할 수 있다.
- Q-Learning의 기초 원리를 이해한다.
- 강화학습이 실제로 적용되는 분야를 열거할 수 있다.

3.1 비지도학습의 개요 — 레이블 없는 학습

비지도학습(Unsupervised Learning)이란 정답(레이블)이 없는 데이터에서 숨겨진 구조나 패턴을 스스로 찾아내는 기계학습 방식입니다.

예를 들어, 수천 명의 고객 구매 데이터가 있을 때 "이 고객은 VIP다", "이 고객은 일반 고객이다"라는 레이블 없이도, 구매 패턴이 비슷한 고객끼리 자동으로 묶어주는 것이 비지도학습입니다.

구분	지도학습	비지도학습
레이블	필요	불필요
목표	정답 예측	패턴·구조 발견
대표 알고리즘	회귀, 분류	클러스터링, 차원축소
예시	스팸 메일 분류	고객 군집화

비지도학습의 주요 분야는 다음과 같습니다.

- **클러스터링(Clustering)**: 비슷한 데이터끼리 그룹으로 묶기
- **차원 축소(Dimensionality Reduction)**: 데이터의 특징을 줄여 핵심만 남기기

- **이상 탐지(Anomaly Detection)**: 정상 패턴에서 벗어난 데이터 찾기
- **연관 규칙(Association Rule)**: 함께 발생하는 패턴 찾기

> 💡 *비지도학습은 레이블 작업 비용이 없어 실무에서 매우 광범위하게 사용됩니다. 인터넷에 쌓이는 방대한 비레이블 데이터를 활용할 수 있다는 점이 큰 장점입니다.*

3.2 클러스터링 — K-Means, DBSCAN, 계층적 클러스터링

클러스터링(Clustering)은 데이터를 비슷한 것끼리 묶어 **군집(Cluster)**을 형성하는 기법입니다. 사람이 직접 분류 기준을 정하지 않아도, 알고리즘이 데이터의 유사성을 기반으로 자동으로 그룹을 나눕니다.

K-Means 클러스터링

K-Means는 가장 널리 쓰이는 클러스터링 알고리즘입니다. 사용자가 군집 수 K를 미리 지정하면, 알고리즘이 데이터를 K개의 군집으로 나눕니다.

작동 원리:

1. K개의 중심점(**센트로이드, Centroid**)을 임의로 초기화합니다.
2. 각 데이터를 가장 가까운 센트로이드에 할당합니다.
3. 각 군집의 평균값으로 센트로이드를 재계산합니다.
4. 센트로이드가 더 이상 변하지 않을 때까지 2~3을 반복합니다.

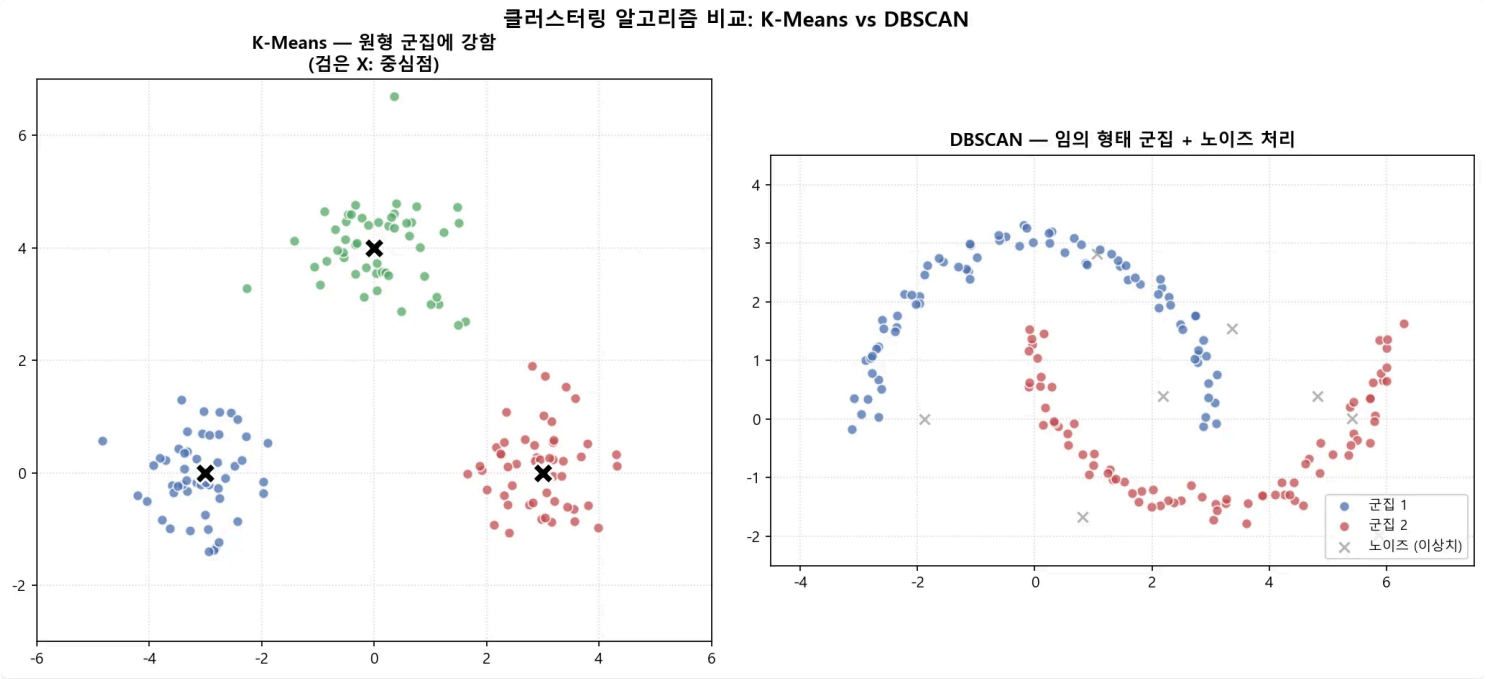
> ⚠️ *K-Means는 K를 미리 정해야 하고, 원형 군집에 적합합니다. 불규칙한 형태의 군집이나 이상치가 많은 데이터에는 성능이 떨어질 수 있습니다.*

DBSCAN

DBSCAN(Density-Based Spatial Clustering of Applications with Noise)은 밀도 기반 클러스터링 알고리즘입니다. K처럼 군집 수를 미리 정하지 않아도 되며, 불규칙한 모양의 군집도 잘 찾아냅니다.

핵심 개념은 "가까운 데이터가 충분히 많이 모여 있으면 하나의 군집"이라는 것입니다. 밀도가 낮은 곳에 홀로 있는 데이터는 **노이즈(Noise)**로 처리합니다.

구분	K-Means	DBSCAN
군집 수	사전 지정 필요	자동 결정
군집 형태	원형에 적합	임의 형태 가능
이상치 처리	취약	노이즈로 처리
속도	빠름	데이터 크기에 따라 느려질 수 있음



계층적 클러스터링

계층적 클러스터링(Hierarchical Clustering)은 데이터를 계층 구조로 묶어가는 방식입니다. 결과를 **덴드로그램(Dendrogram)**이라는 나무 형태의 그림으로 표현하여 어떤 데이터가 먼저 묶이는지 한눈에 볼 수 있습니다.

- **병합(Agglomerative) 방식**: 각 데이터를 개별 군집으로 시작해 점점 합쳐나감 (아래에서 위로)
- **분할(Divisive) 방식**: 전체를 하나의 군집으로 시작해 점점 나눠나감 (위에서 아래로)

> 📌 *계층적 클러스터링은 K를 미리 정하지 않아도 되고, 덴드로그램으로 군집 구조를 시각적으로 파악할 수 있다는 장점이 있습니다. 다만 데이터가 많아질수록 계산 비용이 급격히 증가합니다.*

3.3 차원 축소 — PCA(주성분 분석)

데이터의 특징(Feature) 수가 너무 많으면 학습이 느려지고, 시각화도 어려워집니다. 이를 해결하기 위해 **차원 축소 (Dimensionality Reduction)** 기법을 사용합니다.

PCA(Principal Component Analysis, 주성분 분석)는 대표적인 차원 축소 알고리즘입니다. 데이터의 분산(정보)을 최대한 보존하면서 새로운 축(**주성분, Principal Component**)으로 데이터를 변환하여 차원을 줄입니다.

> 💡 *예를 들어 키와 몸무게, 허리둘레 3가지 특징이 있을 때, PCA는 이 3가지 정보를 최대한 담는 2개(또는 1개)의 새로운 축을 만들어 냅니다.*

PCA의 주요 활용:

- 고차원 데이터를 2D/3D로 줄여 시각화
- 머신러닝 모델 학습 속도 향상
- 중복되거나 불필요한 특징 제거 (노이즈 감소)

PCA 핵심 파라미터:

- `n_components` : 축소할 차원 수 (예: 2이면 2차원으로 축소)
- `explained_variance_ratio_` : 각 주성분이 전체 분산 중 몇 %를 설명하는지 나타냄

> ⚠️ *PCA로 축소된 축은 원래 특징의 의미를 잃어버리므로, "첫 번째 주성분이 무엇을 의미하는지" 직관적으로 해석하기 어렵습니다.*

3.4 이상탐지와 연관규칙

이상탐지

이상탐지(Anomaly Detection)는 정상 패턴에서 크게 벗어난 데이터(**이상치, Anomaly**)를 찾아내는 기법입니다. 대부분의 데이터가 정상이고 이상치는 매우 드물기 때문에 레이블 없이 학습하는 비지도학습 방식이 많이 사용됩니다.

실제 활용 사례:

- 금융: 신용카드 부정 사용(사기) 탐지

- 제조: 공장 설비의 이상 징후 감지
- 네트워크: 사이버 침입 탐지
- 의료: 비정상적인 진단 수치 감지

대표 알고리즘으로는 **Isolation Forest**, **One-Class SVM**, **Autoencoder** 등이 있습니다.

연관규칙

연관규칙(Association Rule)은 대량의 거래 데이터에서 "함께 자주 등장하는 항목들의 규칙"을 찾는 기법입니다. 대형마트의 장바구니 분석이 가장 유명한 예시입니다.

> 💡 *"기저귀를 산 고객은 맥주도 함께 산다"는 실제 마트 데이터 분석에서 발견된 유명한 연관규칙 사례입니다.*

주요 지표:

- **지지도(Support)**: 전체 거래 중 해당 조합이 등장하는 비율
- **신뢰도(Confidence)**: A를 샀을 때 B도 살 확률
- **향상도(Lift)**: 연관규칙이 우연보다 얼마나 강한지 나타내는 값 (1보다 크면 양의 연관)

대표 알고리즘: **Apriori**, **FP-Growth**

3.5 강화학습의 개요 — 에이전트, 환경, 상태, 행동, 보상

강화학습(Reinforcement Learning, RL)은 **에이전트(Agent)**가 **환경(Environment)**과 상호작용하면서 시행착오를 통해 최대 **보상(Reward)**을 얻는 방법을 스스로 학습하는 방식입니다.

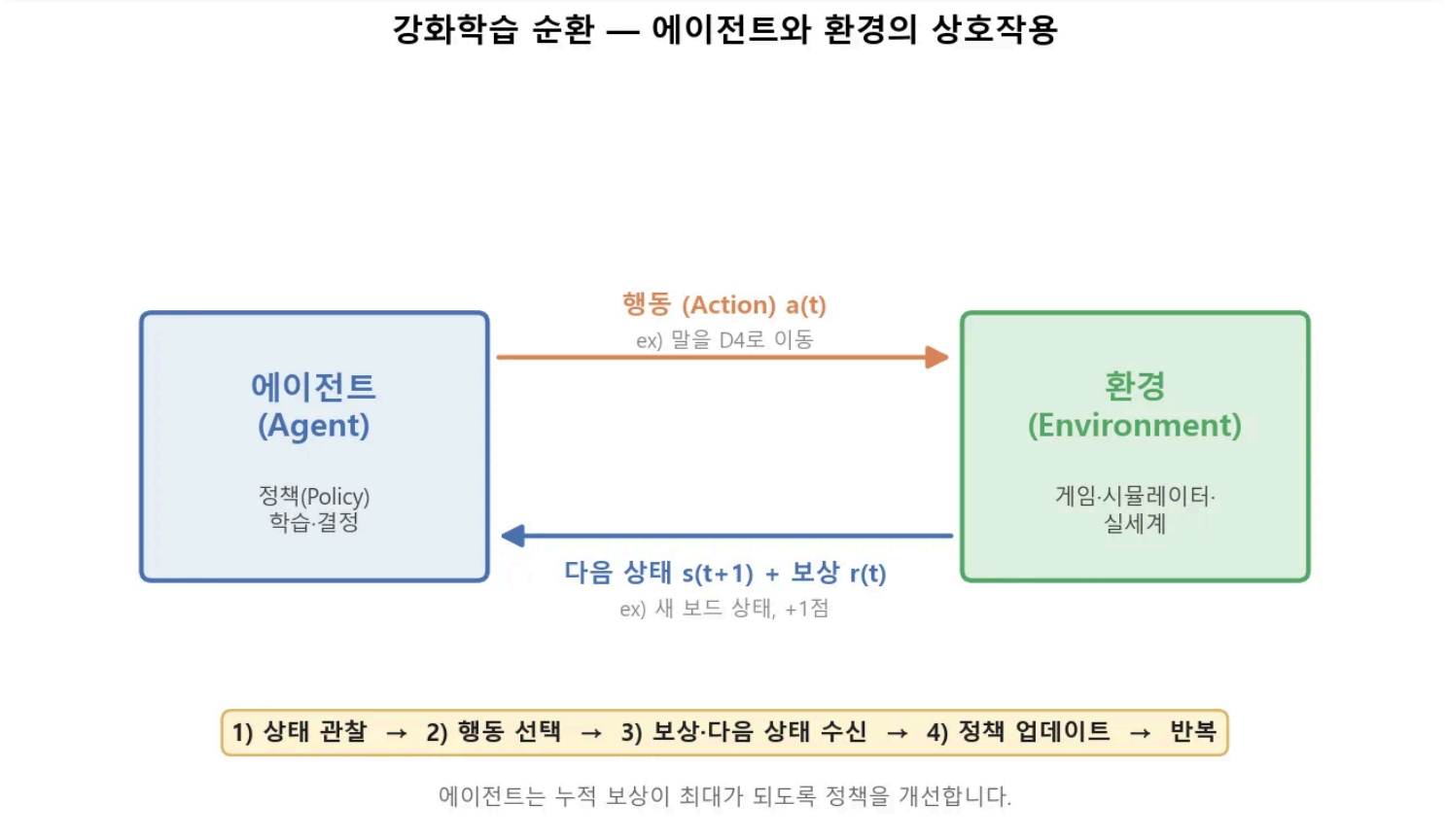
게임을 처음 배우는 아이를 떠올려보세요. 규칙을 누가 알려주지 않아도, 여러 번 플레이하면서 어떤 행동이 점수를 올리는지 스스로 터득하는 것과 같은 원리입니다.

강화학습의 핵심 구성요소:

구성요소	설명	예시 (체스 게임)
에이전트(Agent)	학습하고 행동하는 주체	AI 체스 플레이어
환경(Environment)	에이전트가 상호작용하는 세계	체스 보드
상태(State)	현재 환경의 상황	현재 말의 위치
행동(Action)	에이전트가 취할 수 있는 선택	말을 어디로 이동할지
보상(Reward)	행동의 결과로 받는 피드백	상대 말을 잡으면 +1, 지면 -1
정책(Policy)	상태에서 행동을 선택하는 전략	"이 상황에서는 이 말을 움직이자"

강화학습의 학습 흐름:

1. 에이전트가 현재 **상태(State)**를 관찰합니다.
2. **정책(Policy)**에 따라 **행동(Action)**을 선택합니다.
3. 환경이 **보상(Reward)**과 다음 **상태(Next State)**를 반환합니다.
4. 에이전트는 누적 보상이 최대가 되도록 정책을 업데이트합니다.
5. 1~4를 반복합니다.



> 📌 *강화학습은 지도학습처럼 정답을 알려주지 않습니다. 오직 보상 신호만으로 좋은 행동과 나쁜 행동을 스스로 구분하며 학습합니다.*

3.6 Q-Learning 기초 개념

Q-Learning은 강화학습의 가장 기초적인 알고리즘 중 하나입니다. 각 (상태, 행동) 쌍에 대해 **Q값(Q-value)**이라는 "기대 누적 보상"을 계산하고, 이 Q값을 기반으로 행동을 선택합니다.

Q값을 저장하는 표를 **Q-테이블(Q-Table)**이라고 합니다. 행은 상태(State), 열은 행동(Action)을 나타내며, 학습이 진행될수록 Q값이 점점 정확해집니다.

Q값 업데이트 공식 (벨만 방정식):

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- `s` : 현재 상태, `a` : 선택한 행동
- `r` : 받은 보상
- `s'` : 다음 상태
- `α` (알파): 학습률 — 새 정보를 얼마나 빠르게 반영할지
- `γ` (감마): 할인율 — 미래 보상을 현재 가치로 얼마나 환산할지

> 💡 *할인율 γ가 1에 가까울수록 먼 미래의 보상도 중요하게 여기고, 0에 가까울수록 당장의 보상만 중요하게 여깁니다.*

Q-Learning의 한계와 발전:

Q-테이블은 상태와 행동의 수가 적을 때는 잘 동작하지만, 체스나 바둑처럼 상태 공간이 거대할 때는 테이블 자체를 만들 수 없습니다. 이 한계를 극복하기 위해 딥러닝을 결합한 **DQN(Deep Q-Network)**이 등장했습니다.

3.7 강화학습의 실제 적용 사례

강화학습은 다양한 분야에서 놀라운 성과를 거두고 있습니다.

게임

- **AlphaGo / AlphaZero (DeepMind)**: 바둑 세계 챔피언을 이긴 AI. 강화학습으로 자기 자신과 대국하며 스스로 실력을 키웠습니다.
- **OpenAI Five**: 팀 협동 게임 Dota 2에서 프로 팀을 이긴 AI.
- **Atari 게임**: DQN이 픽셀만 보고 수십 가지 아타리 게임을 사람 수준 이상으로 플레이했습니다.

로봇공학

로봇 팔이 물체를 집는 방법, 두발로 걷는 방법 등을 사람이 일일이 코딩하지 않고 강화학습으로 스스로 터득합니다. Boston Dynamics의 로봇 등이 대표적인 연구 사례입니다.

자율주행

차선 변경, 교차로 통과 등 복잡한 교통 상황에서의 의사결정에 강화학습이 활용됩니다. 시뮬레이터 환경에서 수백만 번의 가상 주행을 통해 안전한 주행 정책을 학습합니다.

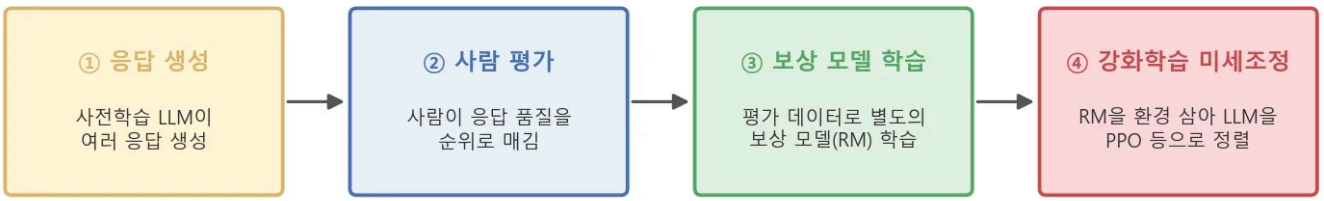
RLHF — 대형 언어 모델의 핵심 기술

RLHF(Reinforcement Learning from Human Feedback, 인간 피드백 강화학습)는 ChatGPT, Claude와 같은 대형 언어 모델(LLM)을 사람의 선호에 맞게 정렬(Alignment)하는 핵심 기술입니다.

RLHF 학습 과정:

1. 사전 학습된 언어 모델이 여러 응답을 생성합니다.
2. 사람이 응답의 품질을 평가하고 순위를 매깁니다.
3. 이 데이터로 **보상 모델(Reward Model)**을 학습합니다.
4. 보상 모델을 환경으로 삼아 언어 모델을 강화학습으로 미세조정합니다.

RLHF (인간 피드백 강화학습) 4단계 흐름



결과: ChatGPT·Claude·Gemini 같은 LLM이 "사람이 선호하는 답"을 생성하도록 정렬(Alignment)됩니다.

> 📌 *RLHF 덕분에 AI가 단순히 다음 단어를 예측하는 것을 넘어, 사람이 원하는 유용하고 안전한 응답을 생성하도록 발전할 수 있었습니다.*

[실습] K-Means 클러스터링으로 데이터 군집화하기

이제 Python으로 K-Means 클러스터링을 직접 실습해 보겠습니다. scikit-learn의 내장 데이터셋을 활용하여 데이터를 군집화하고 시각화합니다.

환경 준비

다음 라이브러리가 필요합니다. 설치되어 있지 않다면 아래 명령어로 설치하세요.

```
pip install scikit-learn matplotlib numpy
```

실습 1: 기본 K-Means 클러스터링

먼저 샘플 데이터를 생성하고 K-Means로 군집화합니다.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# 1. 샘플 데이터 생성 (300개 데이터, 4개 군집)
X, y_true = make_blobs(n_samples=300, centers=4,
                       cluster_std=0.60, random_state=42)

# 2. K-Means 모델 생성 및 학습
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
kmeans.fit(X)

# 3. 군집 레이블과 중심점 확인
labels = kmeans.labels_
centers = kmeans.cluster_centers_

print("군집 레이블 (처음 10개):", labels[:10])
print("군집 중심점:\n", centers)

# 4. 결과 시각화
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis',
            s=50, alpha=0.8, label='데이터')
plt.scatter(centers[:, 0], centers[:, 1],
            c='red', marker='X', s=200, label='중심점')
plt.title('K-Means 클러스터링 결과')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.tight_layout()
plt.show()
```

실행하면 4개의 군집이 서로 다른 색으로 표시되고, 빨간 X가 각 군집의 중심점입니다.

실습 2: 최적의 K 찾기 — 엘보우 방법

K-Means에서 가장 중요한 것은 적절한 K를 선택하는 것입니다. **엘보우 방법(Elbow Method)**은 K를 바꿔가며 **관성(Inertia)** — 각 데이터와 소속 중심점 사이의 거리 합 — 을 계산하여 최적의 K를 찾습니다.

```
# 엘보우 방법으로 최적 K 탐색
inertia_list = []
k_range = range(1, 11)

for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    inertia_list.append(km.inertia_)

# 엘보우 그래프 시각화
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia_list, marker='o', color='steelblue')
plt.axvline(x=4, color='red', linestyle='--', label='최적 K=4')
plt.title('엘보우 방법 — 최적 K 찾기')
plt.xlabel('군집 수 (K)')
plt.ylabel('관성 (Inertia)')
plt.legend()
plt.tight_layout()
plt.show()
```

그래프에서 기울기가 급격히 완만해지는 "팔꿈치" 지점이 최적의 K입니다.

> 💡 *위 예시에서는 K=4 근방에서 기울기가 꺾이는 것을 확인할 수 있습니다. 실제 데이터에서는 명확한 엘보우가 나타나지 않을 수도 있으며, 이때는 실루엣 계수(Silhouette Score) 등 다른 지표를 함께 사용합니다.*

실습 3: 실제 데이터에 적용 — 붓꽃(Iris) 데이터셋

이번에는 유명한 붓꽃 데이터셋에 K-Means를 적용하고 PCA로 시각화합니다.

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. 데이터 로드
iris = load_iris()
X_iris = iris.data          # 4개 특징 (꽃잎 길이/너비 등)
y_iris = iris.target        # 실제 종 (시각화 비교용)

# 2. 데이터 표준화 (K-Means는 거리 기반이므로 스케일링 필수)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_iris)

# 3. K-Means 클러스터링 (K=3, 붓꽃 종류가 3가지)
kmeans_iris = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_iris.fit(X_scaled)
labels_iris = kmeans_iris.labels_

# 4. PCA로 2차원 축소 후 시각화
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# K-Means 군집 결과
axes[0].scatter(X_pca[:, 0], X_pca[:, 1],
                c=labels_iris, cmap='Set1', s=60, alpha=0.8)
axes[0].set_title('K-Means 군집화 결과 (PCA 2D)')
axes[0].set_xlabel('PC1')
axes[0].set_ylabel('PC2')

# 실제 정답 레이블
axes[1].scatter(X_pca[:, 0], X_pca[:, 1],
                c=y_iris, cmap='Set1', s=60, alpha=0.8)
axes[1].set_title('실제 붓꽃 종류 (정답)')
axes[1].set_xlabel('PC1')
axes[1].set_ylabel('PC2')

print(f"PCA 설명 분산 비율: {pca.explained_variance_ratio_}")
print(f"PC1+PC2 누적 설명력: {sum(pca.explained_variance_ratio_):.1%}")
```

```
plt.tight_layout()
plt.show()
```

> 📌 *K-Means 결과와 실제 정답을 나란히 비교해보세요. 레이블 없이도 실제 종 분류와 상당히 유사한 군집을 만들어 낸 것을 확인할 수 있습니다.*

마치며

이번 챕터에서는 레이블 없는 데이터에서 패턴을 찾는 비지도학습과, 시행착오를 통해 스스로 학습하는 강화학습의 핵심 개념을 살펴보았습니다.

핵심 요약:

주제	핵심 내용
비지도학습	레이블 없이 데이터의 패턴·구조를 스스로 발견
K-Means	K개 군집 중심을 반복 업데이트하여 군집화
DBSCAN	밀도 기반 군집화, 이상치 처리 가능
PCA	분산을 최대 보존하며 차원을 축소
이상탐지	정상 패턴 학습 후 이탈 데이터 감지
연관규칙	함께 등장하는 항목의 규칙 발견 (장바구니 분석)
강화학습	에이전트가 환경과 상호작용하며 보상을 최대화
Q-Learning	Q-테이블로 (상태, 행동)의 가치를 학습
RLHF	인간 피드백으로 LLM을 사람의 선호에 정렬

다음 챕터 예고:

Chapter 4에서는 인간의 뇌를 모방한 **딥러닝(Deep Learning)**의 기초를 배웁니다. 인공 신경망의 구조와 학습 원리, 그리고 이미지 인식에 탁월한 **CNN(합성곱 신경망)**의 개념을 살펴보겠습니다.